16. Scope: Local vs. Global Variables

Created by Li Huanyu, last modified by suncaper on Mar 09, 2026

Introduction

In the previous lesson, we learned how to encapsulate our code into **Functions**. Functions are incredibly powerful, but they operate under a strict rule: *What happens inside a function, stays inside the function.* This concept is known as **Scope**. Scope determines where in your script a specific variable can be seen or used. If you create a variable inside a function, the rest of your script has no idea it exists.

In Data Engineering, misunderstanding scope is a leading cause of pipeline bugs. When you move from writing scripts on a single laptop to distributing functions across hundreds of Spark servers, knowing exactly where your data lives is critical.

In this lesson, we will cover:

- **Local Scope:** Variables trapped inside functions.
- **Global Scope:** Variables accessible everywhere.
- The danger of the `global` keyword.
- Why "Stateless" functions are the gold standard in Big Data.

Part 1: Local Scope (The Vegas Rule)

When you create a variable *inside* a function, it is **Local**. It is born when the function is called, and it is completely destroyed the moment the function finishes running.

If you try to print a local variable from outside the function, Python will throw a `NameError` and crash.

```
def process_record():
    # This variable only exists INSIDE this function
    record_status = "Processed"
    return True

process_record()

# ❌ Incorrect: Python has no idea what 'record_status' is down here.
# print(record_status)
# <--- NameError: name 'record_status' is not defined


The Fix: If you need that data outside the function, you must return it (as we learned in the
```

Part 2: Global Scope

A variable created in the main body of your script (outside of any function) is **Global**. Global variables can be *read* by any function in your script. They are typically used for pipeline configurations, constants, or credentials.

```
# A Global Variable (usually written in ALL_CAPS by convention)
ENVIRONMENT = "PRODUCTION"
```

```
def check_env():
    # The function can 'see' the global variable
    print("Running pipeline in: " + ENVIRONMENT)

check_env() # Output: Running pipeline in: PRODUCTION
```

> **The `global` Keyword Trap** ⚠️
>
> While functions can *read* global variables, Python actively prevents functions from *changing* them. If you try to update a global counter inside a function, Python will just create a new, temporary local variable instead.
>
> While you can force Python to overwrite a global variable using the `global` keyword, **Data Engineers strictly avoid this**. Modifying global state makes your pipelines unpredictable and incredibly difficult to debug.

Part 3: The "Big Data" Context (Why do we care?)

1. **Spark Architecture (Driver vs. Worker):** In Apache Spark, your main script runs on a "Driver" node, but your functions are sent to dozens of "Worker" nodes to process the actual data. If your function relies on a Global Variable, that variable lives on the Driver. The Workers cannot see it, and your pipeline will crash. Spark requires special tools (like Broadcast Variables) to share global data safely.
2. **Pure Functions (Statelessness):** Modern data pipelines rely on "Pure Functions." A pure function relies *only* on the parameters passed into it and returns a clean result, without ever touching global variables. This guarantees that if you run the exact same data through the function 10,000 times, you will get the exact same result every single time.
3. **Thread Safety:** When processing massive datasets in parallel (multiple tasks running at the exact same time), if two different tasks try to update the same global variable simultaneously, the data becomes corrupted.

Part 4: Hands-on Exercise (Pipeline Configuration)

Let's simulate a script that uses a global configuration variable safely. We will define our database settings globally, and our local functions will read them to know where to send the data.

1. Open PyCharm.
2. Create a new file named `scope_practice.py`.
3. Type the following code:

```
# ---------------------------------------------------------
# Big Data Simulation: Global Configs & Local Logic
# ---------------------------------------------------------

"""
Scenario: We have a global configuration for our target database.
Our extraction function will use local variables to process data,
but will read the global config to know where to route it.
"""

# 1. GLOBAL SCOPE: Configuration Variables
TARGET_DATABASE = "snowflake_prod_db"
BATCH_SIZE = 1000
```

```
# 2. DEFINE FUNCTION: Using Local Scope
def process_batch(batch_id):
    # 'records_processed' is strictly LOCAL.
    # It resets every time this function is called.
    records_processed = BATCH_SIZE

    print("Batch [" + str(batch_id) + "] cleaned locally.")

    # The function safely reads the GLOBAL variable 'TARGET_DATABASE'
    print("Routing " + str(records_processed) + " records to: " + TARGET_DATABASE)

    return True

# 3. EXECUTE PIPELINE
print("--- Starting ETL Run ---")

# Calling the function multiple times
process_batch(1)
process_batch(2)

print("--- ETL Complete ---")

# ❌ If we tried to print(records_processed) here, it would crash!


Run the code. Output:

--- Starting ETL Run ---
Batch [1] cleaned locally.
Routing 1000 records to: snowflake_prod_db
Batch [2] cleaned locally.
Routing 1000 records to: snowflake_prod_db
--- ETL Complete ---
```

What happens?

- We define `TARGET_DATABASE` and `BATCH_SIZE` in the global scope.
- Inside `process_batch()`, we create a local variable `records_processed`.
- The function easily reads the global variables to format its output.
- The local variable `records_processed` is created and destroyed twice (once for each function call), keeping our memory footprint clean.

Summary

- **Local Scope:** Variables created inside a function. They cannot be accessed outside the function.
- **Global Scope:** Variables created at the top level of a script. Functions can read them, but shouldn't modify them.
- **Pass Data, Don't Share State:** Instead of modifying global variables, pass data into functions via parameters and extract data using `return`.
- **Data Pipelines depend on Scope:** Distributed systems like PySpark isolate functions on different servers. Relying on global variables in Big Data will cause your distributed jobs to fail.

No labels